

A Labwork 3 Report

Luc Lacotte

September 30, 2026

1 Introduction

This labwork is about GPU programming with CUDA. The goal is to implement a grayscale filter for images using both CPU and GPU, and compare their performance.

2 Load an image from file

To load an image from file, we use `imread` from `matplotlib.pyplot`. I will use the default background image of Windows XP for this labwork.

```
from matplotlib import pyplot as plt
img = plt.imread("windowsxp.jpg")
```



3 Reshape the image into 1D array of RGB

For this part, we use the `reshape` function with the size of the image and the number of channels in the parameters.

```
reshaped_img = img.reshape((img.shape[0] * img.shape[1], 3))
```

4 Implement grayscale using CPU

4.1 CPU implementation

For that, we use a for loop, based on your provided GPU function but edited to work on CPU.

```

def grayscaleCPU(image):
    for i in range(image.shape[0]):
        gray = np.uint8((int(image[i, 0]) + int(image[i, 1]) + int(image[i, 2])) / 3)
        image[i, 0] = gray
        image[i, 1] = gray
        image[i, 2] = gray
    return image

t1 = time.time()
grayscaledCPUimg = grayscaleCPU(reshaped_img.copy())
t2 = time.time()
print(f"CPU time: {t2-t1:.6f} seconds")
grayscaledCPUimg = grayscaledCPUimg.reshape((img.shape[0], img.shape[1], 3))
plt.imshow(grayscaledCPUimg)
plt.imsave("windowsxp_grayscaled_cpu.jpg", grayscaledCPUimg)

```

```
>>> CPU time: 14.122947 seconds
```



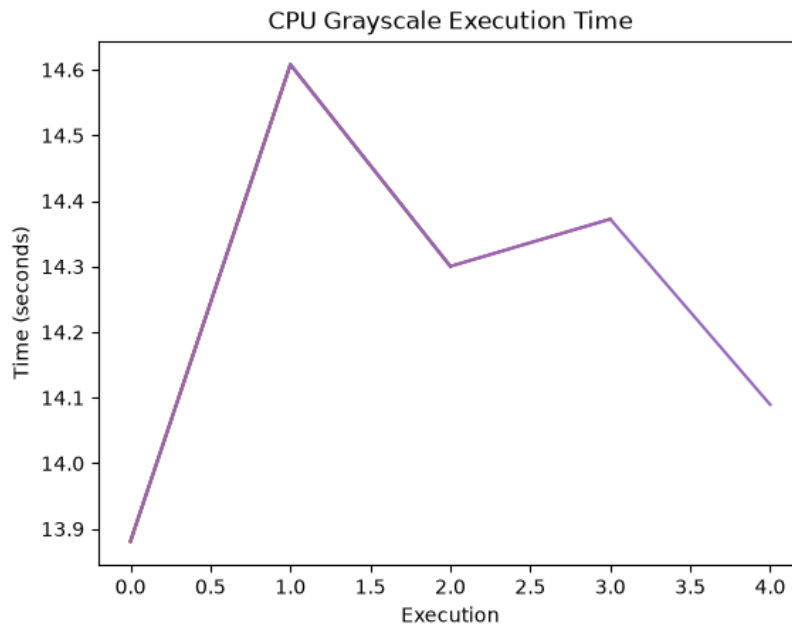
4.2 CPU performance

After multiple executions, the average time for the CPU implementation is around 14–14.5 seconds in my case.

```

times = []
for i in range(5):
    t1 = time.time()
    grayscaledCPUimg = grayscaleCPU(reshaped_img.copy())
    t2 = time.time()
    times.append(t2 - t1)
plt.plot(times)
plt.title("CPU Grayscale Execution Time")
plt.xlabel("Execution")
plt.ylabel("Time (seconds)")
plt.savefig("cpu_grayscale_execution_time.png")

```



5 Implement grayscale using GPU

5.1 GPU implementation

For that, I used the provided function.

```
hostInput = np.zeros(reshaped_img.shape, np.uint8)
devInput = cuda.to_device(reshaped_img)
hostOutput = np.zeros(reshaped_img.shape, np.uint8)
devOutput = cuda.to_device(hostOutput)
pixelCount = reshaped_img.shape[0]
blockSize = 64
gridSize = math.ceil(pixelCount / blockSize)
t1 = time.time()
grayscaledGPUimg = grayscaleGPU[gridSize, blockSize](devInput, devOutput)
t2 = time.time()
hostOutput = devOutput.copy_to_host()
grayscaledGPUimg = hostOutput.reshape((img.shape[0], img.shape[1], 3))
print(f"GPU time: {t2-t1:.6f} seconds")
plt.imshow(grayscaledGPUimg)
plt.imsave("windowxp_grayscaled_gpu.jpg", grayscaledGPUimg)
```

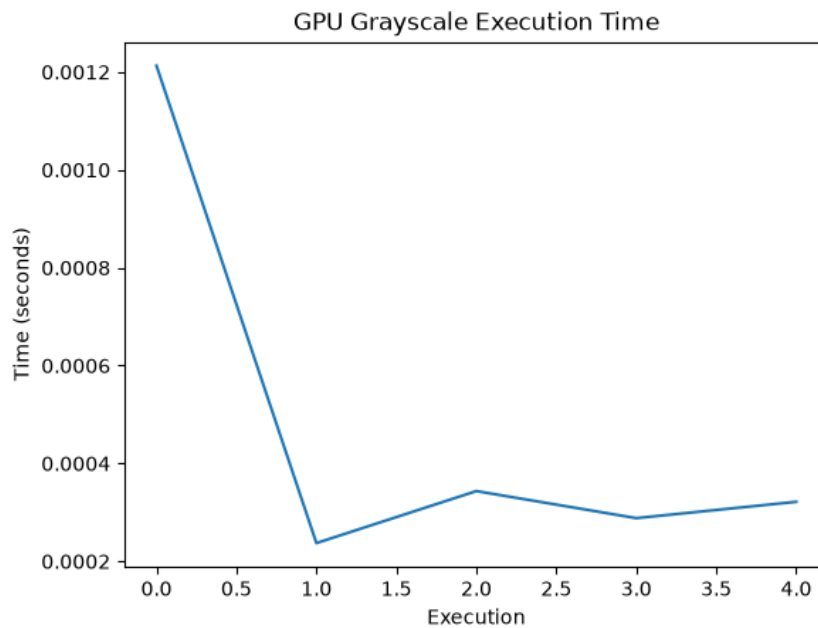
```
>>> GPU time: 0.157658 seconds
```



`math.ceil()` is used to ensure that we have enough blocks to cover all pixels. Otherwise the grid size could be a floating-point number, or the Euclidean division could be rounded down and miss the last pixels.

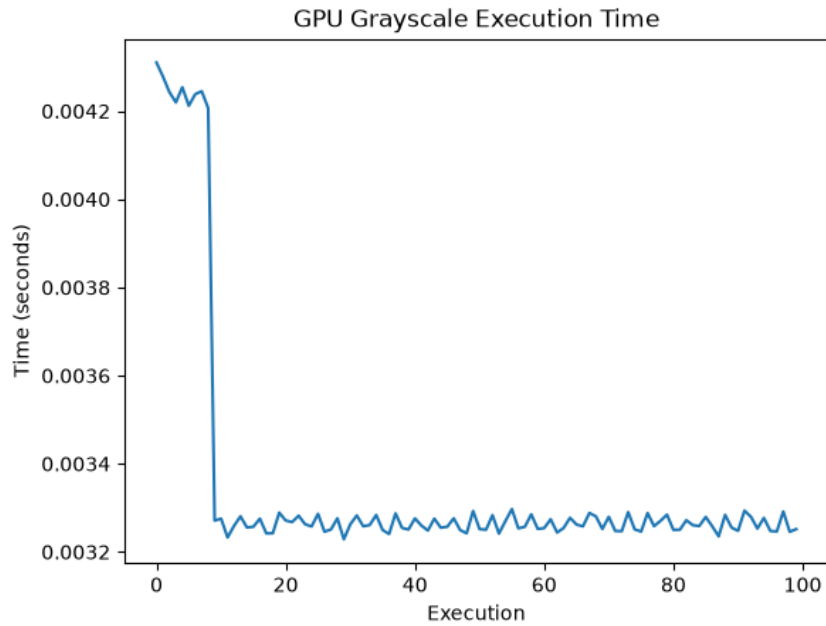
5.2 GPU performance

This gave us a time of 0.15 seconds but this value is not true. When I experimented to get the average time, I noticed that the first execution was always slower than the others. I thought at first that it was because it took into account the compilation time for the first one. This looked like the correct explanation because the other times were really low, around 0.3 milliseconds, but the first values were still off.



But after some research, I found the `cuda.synchronize()` function that blocks the code and waits for the GPU to finish the computation. Because the kernel launch is asynchronous, the time `t2` was taken without waiting for the grayscale image to be fully computed.

So I redid the script using `cuda.synchronize()` and I got an average time of 3.3 milliseconds.

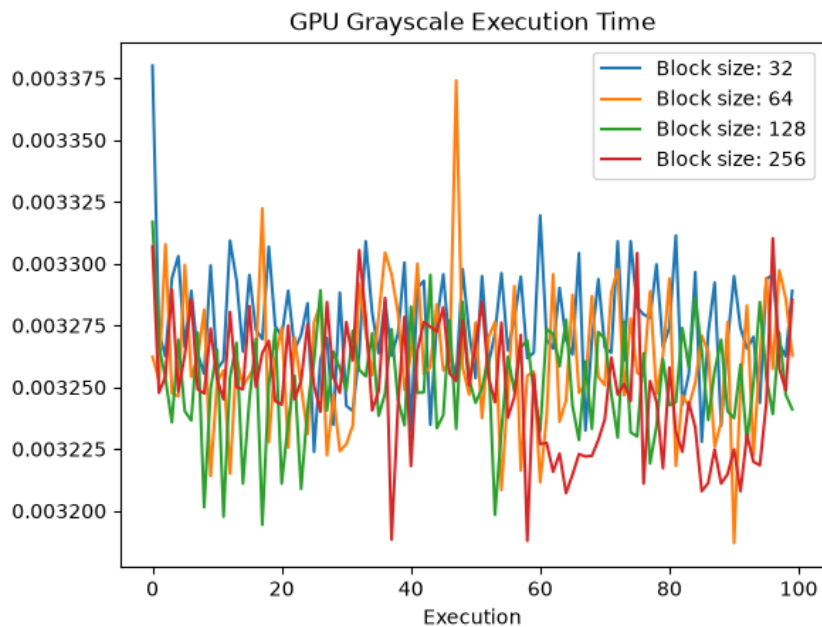


The first few iterations are slower, at around 4.3 milliseconds. I assume it's because the GPU needs to warm up a bit.

5.3 Comparing block sizes

To compare the block sizes, I used the same script as before but I changed the block size to powers of 2 (32, 64, 128 and 256).

These are the results I got:



It seems like the values are not really affected by the block size. I assume that my GPU is limited in computation power and that the block size is not a limiting factor in this case.

6 Dependency problems

I had some problems with the dependencies of the labwork. My GPU drivers were incompatible with the version of numba I had installed, so I had to install a specific version of numba-cuda to

make it work.

```
numba.cuda.cudadrv.driver.LinkerError: [222] Call to cuLinkAddData results
      in CUDA_ERROR_UNSUPPORTED_PTX_VERSION
ptxas application ptx input, line 9; fatal    : Unsupported .version 9.4;
      current version is '9.3'
```

```
pip install "numba-cuda[cu13]"
```